

Глава 19

Распределители памяти

Распределители памяти, введенные в разделе 4.6, представляют собой специальную модель памяти и являются абстракцией, которая используется для преобразования *потребности* в памяти в непосредственное *обращение* к ней. В настоящей главе описываются распределители памяти и соответствующие низкоуровневые функциональные возможности для работы с ней. Подробности изложены в приложении к книге, которое размещено на веб-сайте по адресу <http://www.cppstdlib.com>.

19.1. Использование распределителей памяти с точки зрения прикладного программиста

С точки зрения прикладного программиста использование разных распределителей памяти не должно быть проблемой. Для этого достаточно передать распределитель памяти как шаблонный параметр. Например, следующие операторы создают разные контейнеры и строки с помощью специального распределителя памяти `MyAlloc<>`:

```
// вектор со специальным распределителем памяти
std::vector<int, MyAlloc<int>> v;

// отображение int/float со специальным распределителем памяти
std::map<int, float, std::less<int>,
MyAlloc<std::pair<const int, float>>> m;

// строка со специальным распределителем памяти
std::basic_string<char, std::char_traits<char>, MyAlloc<char>> s;
```

При использовании собственного распределителя памяти, вероятно, целесообразно дать определения нескольких типов. Рассмотрим пример:

```
// специальный строковый тип, использующий специальный распределитель памяти
typedef std::basic_string<char, std::char_traits<char>,
MyAlloc<char>> MyString;

// специальный тип отображения string/string,
// использующий специальный распределитель памяти
typedef std::map<MyString, MyString, std::less<MyString>,
MyAlloc<std::pair<const MyString, MyString>>> MyMap;

// создаем объект указанного типа
MyMap мупар;
```

После принятия стандарта C++11 можно использовать *псевдонимы шаблонов* (alias templates), т.е. объявления шаблонов с помощью операций typedef (см. раздел 3.1.9), позволяющие определить тип распределителя памяти без указания типа элементов.

```
template <typename T>
using Vec = std::vector<T, MyAlloc<T>>; // вектор, использующий собственный
// распределитель памяти
Vec<int> coll; // эквивалент std::vector<int, MyAlloc<int>>
```

При работе с объектами, использующими нестандартный распределитель памяти, разница не заметна.

Для того чтобы проверить, не используют ли два распределителя памяти один и тот же ресурс, предусмотрены операции == и !=. Если оператор == возвращает значение true, то память, выделенную одним распределителем, можно освободить с помощью другого. Для того чтобы получить доступ к распределителю памяти, все типы, параметризованные распределителем памяти, имеют функцию-член get_allocator(). Например:

```
if (mymap.get_allocator() == s.get_allocator()) {
// ОК, mymap и s используют один и тот же или эквивалентные
// распределители памяти
...
}
```

Кроме того, после принятия стандарта C++11 появилось свойство (см. раздел 5.4) для проверки наличия у типа T шаблонного параметра allocator_type, в который можно преобразовать передаваемый распределитель памяти.

```
std::uses_allocator<T, Alloc>::value // true, если тип Alloc допускает
// преобразование в T::allocator_type
```

19.2. Пользовательский распределитель памяти

Распределители памяти имеют интерфейс для выделения и освобождения памяти, а также создания и уничтожения объектов (табл. 19.1). Контейнеры и алгоритмы можно параметризовать распределителями памяти с учетом типа хранящихся в них элементов. Например, можно реализовать распределитель памяти, использующий общую память или отображающий элементы в сохраняемую базу данных.

Таблица 19.1. Основные операции над распределителями памяти

Выражение	Описание
a.allocate(num)	Выделяет память для num элементов
a.construct(p, val)	Инициализирует элемент, на который ссылается указатель p, значением val
a.destroy(p)	Уничтожает элемент, на который ссылается указатель p
a.deallocate(p, num)	Освобождает память от num элементов, на которые ссылается указатель p

Написать свой собственный распределитель памяти не очень сложно. Самым важным аспектом является способ, с помощью которого выделяется или освобождается память. После принятия стандарта C++11 остальные факторы, как правило, можно задать по умолчанию. (До принятия стандарта C++11 их необходимо было реализовать явным образом.) Как пример, рассмотрим распределитель памяти, работающий аналогично распределителю памяти, заданному по умолчанию:

```
// alloc/myalloc11.hpp

#include <cstddef>          // для типа size_t

template <typename T>
class MyAlloc {
public:
    // определения типов
    typedef T value_type;

    // конструкторы
    // - ничего не делают, потому что распределитель памяти не имеет состояния
    MyAlloc () noexcept {
    }
    template <typename U>
    MyAlloc (const MyAlloc<U>&) noexcept {
        // нет состояния для копирования
    }

    // выделяем память для num элементов типа T, но не инициализируем ее
    T* allocate (std::size_t num) {
        // выделяем память с помощью глобальной операции new
        return static_cast<T*> (::operator new (num*sizeof(T)));
    }

    // освобождаем память, на которую ссылается указатель p,
    // от удаленных элементов
    void deallocate (T* p, std::size_t num) {

        // освобождаем память с помощью глобальной операции delete
        ::operator delete(p);
    }
};

// возвращаем признак того, что все специализации данного
// распределителя памяти являются эквивалентными
template <typename T1, typename T2>
bool operator== (const MyAlloc<T1>&,
                 const MyAlloc<T2>&) noexcept {
    return true;
}
template <typename T1, typename T2>
bool operator!= (const MyAlloc<T1>&,
                 const MyAlloc<T2>&) noexcept {
    return false;
}
```

Как следует из этого примера, программист должен реализовать следующие функциональные возможности.

- Определение типа `value_type`, представляющего собой всего лишь тип передаваемого шаблонного параметра.
- Конструктор.
- Шаблонный конструктор, копирующий внутреннее состояние при изменении типа. Отметим, что шаблонный конструктор не подавляет неявное объявление конструктора копирования (см. раздел 3.2).
- Член `allocate()`, выделяющий новую память.
- Член `deallocate()`, освобождающий память, которая больше не нужна.
- Конструктор и деструктор (при необходимости) для инициализации, копирования и очистки внутреннего состояния.
- Операции `==` и `!=`.

Функции `construct()` или `destroy()` реализовывать не обязательно, потому что их реализации по умолчанию обычно работают отлично (используя операцию *new* с размещением для инициализации памяти и вызывая деструктор для ее явной очистки).

Используя эту базовую реализацию, легко создать свой собственный распределитель памяти. Для реализации своей стратегии выделения памяти можно использовать функции `allocate()` и `deallocate()`. Эта стратегия может подразумевать повторное использование памяти вместо ее немедленного освобождения, использование общей памяти, отображение памяти в сегмент объектно-ориентированной базы данных или просто отладку распределителя памяти. Кроме того, программист может предусмотреть свои конструкторы и деструктор для выделения и освобождения памяти вместо функций `allocate()` и `deallocate()`. До принятия стандарта C++11 класс распределителя памяти должен был содержать намного больше функций-членов, которые, впрочем, легко можно было написать. Законченный пример, демонстрирующий распределители памяти, можно найти в файле `alloc/myalloc03.hpp`, который описан также в специальном разделе, посвященном распределителям памяти, на веб-сайте <http://www.cppstdlib.com>.

19.3. Использование распределителей памяти с точки зрения разработчика библиотеки

В этом разделе описывается использование распределителей памяти с точки зрения разработчиков контейнеров и других компонентов, способных работать с разными распределителями памяти. Этот раздел частично (с разрешения автора) содержит материал из раздела 19.4 книги Бьярне Страуструпа (Bjarne Stroustrup) *The C++ Programming Language*, 3rd edition (см. [Stroustrup:C++]).

В качестве примера рассмотрим наивную реализацию вектора. Вектор получает свой распределитель памяти как шаблонный параметр или аргумент конструктора и сохраняет его в какой-то области памяти.

```
namespace std {
    template <typename T,
```

```

        typename Allocator = allocator<T> >
class vector {
    ...
private:
    Allocator alloc;        // распределитель
    T* elems;               // массив элементов
    size_type numElems;     // количество элементов
    size_type sizeElems;    // размер памяти для элементов
    ...
public:
    // конструкторы
    explicit vector(const Allocator& = Allocator());
    explicit vector(size_type num, const T& val = T(),
                   const Allocator& = Allocator());
    template <typename InputIterator>
    vector(InputIterator beg, InputIterator end,
           const Allocator& = Allocator());
    vector(const vector<T,Allocator>& v);
    ...
};
}

```

Отметим, что, строго говоря, тип элементов по стандарту C++11 должен быть следующим:

```
allocator_traits<Allocator>::pointer elems;
```

Как и свойства итератора (см. раздел 9.5), *свойства распределителя памяти* были изобретены для того, чтобы они служили общим интерфейсом для обобщенного кода, работающего с распределителями памяти. Они поддерживают такие типы, как `pointer`, и такие операции, как `allocate()`, `construct()`, `destroy()` и `deallocate()`.

Второй конструктор, инициализирующий вектор `num` элементами со значением `val`, должен быть реализован следующим образом:

```

namespace std {
    template <typename T, typename Allocator>
    vector<T,Allocator>::vector(size_type num, const T& val,
                               const Allocator& a)
    : alloc(a) // инициализируем распределитель
    {
        // выделяем память
        sizeElems = numElems = num;
        elems = allocator_traits<Allocator>::allocate(alloc, num);

        // инициализируем элементы
        for (size_type i=0; i<num; ++i) {

            // инициализируем i-й элемент
            allocator_traits<Allocator>::construct(alloc, &elems[i], val);
        }
    }
}

```

Отметим, что этот код не полный, потому что в нем нет обработки исключений. В правильной реализации после неудачного создания любого элемента вся выделенная память должна быть освобождена.

Таблица 19.2. Вспомогательные функции для работы с неинициализированной памятью

Выражение	Описание
<code>uninitialized_fill(beg, end, val)</code>	Инициализирует интервал <code>[beg,end)</code> значением <code>val</code>
<code>uninitialized_fill_n(beg, num, val)</code>	Инициализирует <code>num</code> элементов, начиная с позиции <code>beg</code> , значением <code>val</code>
<code>uninitialized_copy(beg, end, mem)</code>	Инициализирует элементы, начиная с позиции <code>mem</code> , элементами интервала <code>[beg,end)</code>
<code>uninitialized_copy_n(beg, num, mem)</code>	Инициализирует <code>num</code> элементов, начиная с позиции <code>mem</code> , элементами, начиная с позиции <code>beg</code> (по стандарту C++11)

Однако для инициализации неинициализированной памяти стандартная библиотека C++ содержит несколько вспомогательных функций (табл. 19.2). С помощью этих функций реализация конструктора становится еще проще:

```
namespace std {
    template <typename T, typename Allocator>
    vector<T,Allocator>::vector(size_type num, const T& val,
                               const Allocator& a)
    : alloc(a) // инициализируем распределитель
    {
        // выделяем память
        sizeElems = numElems = num;
        elems = alloc.allocate(num);

        // инициализируем элементы
        uninitialized_fill_n(elems, num, val);
    }
}
```

Однако по стандарту C++11 функции `uninitialized_fill_n()` и `uninitialized_copy()` применять нельзя, потому что при создании элементов они не используют свойства распределителя памяти. В этом случае управление памятью может быть нарушено пользовательским кодом, в котором определены свойства распределителя памяти и/или типы распределителей памяти, в которых вызов функции `construct()` может приводить в выполнении дополнительных или совершенно других операций.

Функцию-член `reserve()`, резервирующую дополнительную память без изменения количества элементов (см. раздел 7.3.1), можно реализовать следующим образом:

```
namespace std {
    template <typename T, typename Allocator>
    void vector<T,Allocator>::reserve(size_type size)
```

```

{
    // функция reserve() никогда не уменьшает размер памяти
    if (size <= sizeElems) {
        return;
    }

    // выделяем новую память для size элементов
    T* newmem = allocator_traits<Allocator>::allocate(alloc, size);

    // копируем старые элементы в новую память
    ...

    // уничтожаем старые элементы
    for (size_type i=0; i<numElems; ++i) {
        allocator_traits<Allocator>::destroy(alloc, &elems[i]);
    }

    // освобождаем старую память
    allocator_traits<Allocator>::deallocate(alloc, elems, sizeElems);

    // теперь наши элементы записаны в новой памяти
    sizeElems = size;
    elems = newmem;
}
}

```

Отметим еще раз, что этот код является слишком упрощенным. В нем отсутствует сложная часть, относящаяся к копированию элементов в новую память, потому что при этом используются исключения и по возможности должны выполняться перемещающие, а не копирующие операторы.

Простые итераторы

Для обхода и инициализации неинициализированной памяти предназначен класс `raw_storage_iterator`. Написав алгоритм, использующий класс `raw_storage_iterator`, можно инициализировать память значениями, которые являются результатами этого алгоритма.

Например, следующий оператор инициализирует память, на которую ссылается указатель `elems`, значениями из интервала `[x.begin(), x.end())`:

```

copy (x.begin(), x.end(), // источник
      raw_storage_iterator<T*,T>(elems)); // получатель

```

Первый шаблонный параметр (в данном случае `T*`) должен быть итератором вывода для указанного типа элементов. Второй шаблонный параметр (в данном случае `T`) должен быть типом элементов.

Временные буфера

В программах иногда встречаются функции `get_temporary_buffer()` и `return_temporary_buffer()`, предназначенные для обработки неинициализированной памяти,

которая используется в этих функциях как кратковременное и временное хранилище. Функция `get_temporary_buffer()` может вернуть меньше памяти, чем ожидалось. Таким образом, функция `get_temporary_buffer()` возвращает пару, содержащую адрес памяти и ее размер (количество элементов). Рассмотрим пример использования этих функций:

```
void f()
{
    // выделяем память для num элементов типа MyType
    pair<MyType*,std::ptrdiff_t> p
    = get_temporary_buffer<MyType>(num);
    if (p.second == 0) {
        // невозможно выделить память для элементов
        ...
    }
    else if (p.second < num) {
        // невозможно выделить память для num элементов
        // однако ее следует не забыть освободить
        ...
    }

    // обработка
    ...

    // освобождаем временно выделенную память, если она есть
    if (p.first != 0) {
        return_temporary_buffer(p.first);
    }
}
```

Но с помощью функций `get_temporary_buffer()` и `return_temporary_buffer()` довольно сложно написать безопасный с точки зрения исключений код, поэтому они, как правило, не используются в реализациях библиотеки.